

RUPTURE — V5.2

Rapport technique officiel — audit du plafond de crise à 98%, correction de l'équilibrage, redesign musical et validation complète

Dépôt : `scuillerjack-oss/rupture-v0` · Branche : `main`
Commits livrés : `4de7e3f` (moteur/équilibre) puis `8e0b45c` (audio)
URL permanente de bêta : <https://scuillerjack-oss.github.io/rupture-v0/>

1. État initial trouvé

Avant toute modification, le dépôt était à jour au commit `cd2d612` (dernier commit de la phase V5.1, déjà déployé avec succès — run GitHub Actions `34780471840`, conclusion *success*). Aucune reconstruction depuis zéro n'a été effectuée : tout le travail de cette phase part de cet état, avec ses acquis (gameplay validé, UI mobile, PWA, sauvegardes, tests, modèle économique, musique générative V5.1).

Retour de bêta téléphone reçu au début de cette phase, joué en Discrétion → Propagation/Portée → implantation mondiale → Dangerosité maximale, Résilience laissée à 0 :

Métrique	Valeur rapportée
Issue	Défaite, jour 532
Progression finale / maximum atteint	94% / 98%
Conscience mondiale	72%
Réponse mondiale	100%
Régions touchées / sévères / critiques	18/18 · 18/18 · 18/18
Niveaux finaux (Propagation/Dangerosité/Résilience/Discrétion)	5 / 10 / 0 / 5
Jalons (Conscience / Mobilisation / Contre-mesures / Maîtrise imminente)	J406 / J454 / J502 / J524

Le joueur a explicitement écarté l'hypothèse d'une découverte trop tardive comme cause principale, et a demandé d'auditer en priorité le blocage à 98% de Progression malgré une implantation mondiale totale.

1.1. La distinction « connaissance/présence » vs « perception de dangerosité » existe déjà

Consigne explicite reçue : ne pas ajouter cette mécanique artificiellement, et vérifier d'abord si elle existe déjà dans le moteur réel. Inspection de `src/ui/hud.js` et `src/ui/screens.js` : c'est déjà le cas. **Portée** (reach) mesure la connaissance/présence de l'Anomalie et peut atteindre 100% même à Dangerosité 0. **Progression** (dominance) mesure la perception réelle de danger et reste plafonnée par `dangerosityCapAt(niveau de Dangerosité)` — une Anomalie très répandue mais peu dangereuse

peut donc déjà, dans le moteur existant, être connue sans être perçue comme une menace. Aucun code n'a été ajouté pour ce point : le mécanisme demandé était déjà correctement en place.

2. Cause réelle du blocage à 98%

Sur consigne explicite (« ne code pas nos hypothèses, vérifie le code réel »), la cause a été retrouvée par instrumentation directe du moteur réel (traces tick par tick), pas par supposition.

2.1. Le mécanisme en cause

Dans `src/engine/simulation.js`, chaque territoire a un plafond de crise :

```
dangerosityCap = dangerosityCapAt(niveauDangerosité) // 45 → 100 selon le niveau
suppressionRate = tauxMaxSuppression * mobilisation * (1 - effetRésilience) * tension
crisisCap = dangerosityCap * (1 - suppressionRate)
newCrisis = Math.min(newCrisis, crisisCap)
```

tension croît avec le nombre de jours écoulés (indépendamment du joueur), et mobilisation croît elle aussi avec le temps une fois le seuil de mobilisation franchi. Résultat : **crisisCap diminue** mécaniquement à mesure que la partie avance — y compris après qu'un territoire a déjà atteint une crise élevée.

Le bug réel : l'ancien clamp `Math.min(newCrisis, crisisCap)` laissait ce plafond qui rétrécit *éroder une crise déjà acquise*. Or un commentaire du code, présent depuis V3.1, documente explicitement l'invariant voulu : « une réduction du plafond, jamais une érosion de la crise déjà acquise ». Le code ne respectait plus sa propre règle. C'est exactement le schéma rapporté par le joueur : Progression qui monte jusqu'à 98%, puis se fait tirer vers le bas jusqu'à 94% à la défaite, alors que Dangerosité restait au maximum sur des territoires déjà sévères/critiques.

2.2. Pourquoi Résilience 0 rendait la victoire quasi impossible plutôt que « risquée »

Puisque `crisisCap` baisse avec le temps quel que soit le niveau de Résilience (la Résilience ne fait que réduire l'ampleur de la suppression, pas l'arrêter), une bascule tardive sur Dangerosité maximale arrivait systématiquement après que la fenêtre de plafond haut se soit déjà refermée — la progression butait alors contre un plafond qui ne pouvait plus que rétrécir. Le résultat n'était pas « Résilience 0 = pari risqué », mais « Résilience 0 combinée à une bascule tardive = victoire mathématiquement bloquée », quel que soit le timing du joueur.

3. Fonctionnement de la Réponse mondiale et rôle de la Résilience (après correctif)

La Réponse mondiale reste pilotée par la Conscience (voir V4.1) et progresse par phases jusqu'à la mobilisation, puis aux contre-mesures, puis à la maîtrise. Son *effet mécanique* sur le joueur passe uniquement par `suppressionRate`, qui réduit le plafond de crise atteignable par territoire. La Résilience réduit la part de cet effet (`effetRésilience`) : à Résilience égale, un monde plus mobilisé supprime plus fort ; à mobilisation égale, plus de Résilience supprime moins fort.

Ce mécanisme reste inchangé dans son principe — seul le *calage temporel* entre l'atteinte d'un pic de crise et le moment où la suppression du monde peut effectivement l'entamer a été corrigé (voir §4).

4. Corrections apportées

4.1. Inertie de suppression — `state.suppressionPressure`

Nouveau champ d'état qui rattrape progressivement `mobilizationProgress` au lieu de le refléter instantanément (même principe que les paires `containment/awareness` déjà existantes dans le moteur) :

```
suppressionPressure += (mobilizationProgress - suppressionPressure) *  
suppressionInertiaRate * tension
```

`suppressionInertiaRate = 0.015` (calibré par simulation, voir §6). Cette pression, et non la mobilisation brute, alimente désormais `suppressionRate`. Une bascule tardive sur `Dangerosité` maximale dispose ainsi d'une vraie fenêtre de grâce avant que la suppression n'atteigne sa pleine puissance.

4.2. Plafond à cliquet — `state.crisisCapHighWater`

Second champ d'état, plus direct : un plafond haute-eau, jamais érodé une fois atteint.

```
instantCrisisCap = dangerosityCap * (1 - suppressionRate)  
crisisCapHighWater = Math.max(crisisCapHighWater, instantCrisisCap)  
crisisCap = crisisCapHighWater  
newCrisis = Math.min(newCrisis, crisisCap) // clamp par territoire  
inchangé, simple
```

Ce plafond ne peut désormais que monter ou rester stable, jamais rétrécir — respectant enfin l'invariant documenté depuis V3.1.

4.3. Sauvegardes — SAVE_VERSION 5 → 6

Migration honnête ajoutée dans `migrations.js` : reconstruit `suppressionPressure` à partir de la mobilisation déjà atteinte, et `crisisCapHighWater` à partir de la crise maximale déjà observée dans les territoires sauvegardés — sans jamais offrir un allègement non mérité à une partie en cours.

4.4. Deux pistes explorées puis abandonnées (honnêteté du process)

Piste 1 — plafond conditionnel (`currentCrisis >= crisisCap ? dangerosityCap : crisisCap`) : fonctionnait sur le cas ciblé, mais une fois franchi le plafond supprimé, autorisait un saut direct vers le plafond non supprimé (100). Sur la batterie complète, une heuristique agressive gagnait alors 96% du temps — violation directe de la consigne « Dangerosité ne garantit pas la victoire ». Abandonnée.

Piste 2 — `Math.max(currentCrisis, crisisCap)` comme plafond : prouvée algébriquement figer la progression de façon permanente et totale dès que `crisisCap` passe une première fois sous `currentCrisis` (puisque `min(candidat, currentCrisis) = currentCrisis` exactement, pour toujours). Abandonnée avant même d'être testée en batterie, sur preuve mathématique directe.

Piste 3 — érosion progressive (rapprochement lent du plafond vers la crise plutôt qu'un cliquet dur) : corrigeait le figement brutal de la piste 2, mais était trop faible à l'échelle de temps réelle du jeu — elle faisait retomber le taux de victoire de la stratégie ciblée à ~0% pour tous les niveaux de Résilience. Abandonnée au profit du cliquet (§4.2), plus direct et dont l'effet est vérifiable simplement.

4.5. Limite honnête du correctif retenu

Le cliquet protège aussi un plafond atteint *avant* que la mobilisation n'ait vraiment commencé à agir (quand `suppressionRate` est encore quasi nulle, donc `crisisCap` $\approx$ `dangerosityCap` non supprimé). Une stratégie qui atteint la Dangerosité maximale assez vite verrouille alors un plafond proche de 100% avant que les différences de Résilience ne puissent jouer sur la suppression. Vérifié empiriquement (§6) : pour cette archétype précis (« bascule furtive »), Résilience 0/faible/moyenne donnent des taux de victoire statistiquement identiques (~37-39% sur 90-144 échantillons), y compris avec l'inertie de suppression artificiellement désactivée pour le diagnostic. La Résilience reste en revanche pleinement déterminante dans le reste du paysage stratégique (`resilience-neglige` = 0%, `resilience-precoc` = 0%, contre `reactive-coherente` = 78%) : cette neutralité de la Résilience est une nuance réelle et assumée d'un archétype précis, pas une régression généralisée. Corriger ce point sans réintroduire le problème initial demanderait une mécanique dédiée (par exemple, séparer le verrouillage du plafond de son niveau réel) — jugé

hors-scope pour cette passe au regard de la demande de rester simple et de ne pas sur-architecturer, et documenté ici pour la prochaine bêta.

5. Résumé de fin de partie — correction du bug de timing

Le récapitulatif de fin (`computeStyleAnalysis`, `src/ui/screens.js`) affirmait « Une progression volontairement dangereuse, quitte à alarmer le monde très vite » dès que Dangerosité était le niveau final dominant — sans jamais vérifier *quand* la Conscience mondiale était réellement apparue. C'est exactement le bug rapporté (partie bêta : Conscience jour 406, décrite comme « très vite »).

Correction : le commentaire distingue désormais une bascule réellement précoce (Conscience apparue avant le jour 200) d'une bascule tardive après implantation solide, avec un message dédié citant le jour réel d'apparition de la Conscience. Reproduit et vérifié par un test dédié qui rejoue le scénario bêta exact via le moteur réel (`tests/screens.test.js`).

6. Simulations avant / après

Batterie rejouée intégralement après correctif (`npm run simulate`, 18 échantillons × 3 difficultés par stratégie, moteur réel sans triche). Résultats bruts avant dans `docs/v4.2-simulation-results.json`, après dans `docs/v5.2-simulation-results.json` (script renommé en conséquence). Résultats en difficulté normale :

Stratégie	Avant	Après	Commentaire
passive	0%	0%	toujours perdante par conception (filet de sécurité)
dangerosite-precoce	0%	6%	légère hausse, non alarmante (voir note)
resilience-negligee	0%	0%	Résilience reste déterminante hors archétype de bascule
equilibree-naive	6%	0%	écart dans le bruit d'un run non-seedé à n=18 (1 victoire = 6%)
reactive-coherente	72%	78%	stratégie de référence, cohérente historiquement
furtive-puis-frappe	33%	39%	hausse modérée, cohérente avec le correctif
resilience-precoce	0%	0%	inchangée
achats-aleatoires	17%	17%	inchangée

6.1. Le cas bêta exact — nouvelles stratégies dédiées

Stratégie (Discrétion+Propagation puis bascule)	Résilience	Taux de victoire	Durée moy. (victoire)
---	------------	------------------	-----------------------

furtive-bascule-resilience0	0	39% (0% avant correctif — mathématiquement impossible)	491 j
furtive-bascule-resilience-faible	jusqu'à 3	33%	488 j
furtive-bascule-resilience-moyenne	jusqu'à 6	39%	489 j

Confirmé par lots multi-échantillons (90 à 144 parties par variante, moteur non-seedé — chaque lot moyenné pour absorber le bruit intrinsèque de `Math.random()`) : la stratégie du joueur peut désormais **gagner ou perdre** selon le timing exact de la bascule, au lieu d'être plafonnée mathématiquement à la défaite.

6.2. Stress-tests (recherche volontaire de casse de l'équilibrage)

Risque recherché	Constaté	Verdict
Stratégie dominante	Maximum observé : 78% (reactive-coherente)	absent
Stratégie toujours perdante malgré un jeu actif	Seules les stratégies délibérément mauvaises (passive, résilience négligée/précoce...) restent à 0% — par conception	acceptable
Stagnation 98/99%	Plus observée sur aucune stratégie testée après correctif	corrigé
Réponse mondiale trop forte/faible	Hierarchie des taux de victoire proche des références historiques (V4.2)	stable
Discrétion abusive	Aucune stratégie 100% Discrétion ne dépasse les stratégies équilibrées	absent
Résilience rendue obligatoire	furtive-bascule-resilience0 gagne autant que ses variantes à Résilience plus haute	nuance documentée en §4.5
Dangerosité rendue inutile	Structurellement requise : la Progression reste plafonnée à 45 sans Dangerosité, victoire impossible	absent
Victoire automatique après implantation mondiale (Portée 100%)	Aucune corrélation observée entre Portée=100% et Progression=100% — Portée et Progression restent des axes distincts	absent

7. Musique — de la nappe générative au thème identifiable

Retour bêta explicite sur la V5.1 : l'ambiance générative continue (nappe d'accords + notes piochées au hasard, La mineur naturel) restait perçue comme trop sombre/angoissante, et de toute façon ce n'était pas la demande — une vraie petite mélodie identifiable était souhaitée, pas un drone ambient.

7.1. Choix retenus

- **Thème fixe et reconnaissable**, pas une génération procédurale note par note : un thème principal de 8 notes (contour en arche) et une phrase réponse de 6 notes (descendante, volontairement

ouverte), qui alternent à chaque apparition.

- **La Dorien plutôt que La mineur naturel** : le 6e degré rehaussé (Fa# au lieu de Fa) donne une couleur pensive/curieuse plutôt que triste/sombre en théorie modale usuelle — c'est le levier théorique choisi pour répondre directement au ressenti « angoissant » sans changer le tempo ni la densité.
- **Un seul soutien harmonique très doux** (note grave tenue, jamais un accord complet), présent uniquement pendant la durée du thème — plus aucun son continu entre deux apparitions.
- **De vraies plages de silence** (35-65 secondes) entre chaque apparition, conformément à la demande explicite de pouvoir tourner en fond sans fatiguer.
- **Humanisation légère** (tempo, durée, volume variés à chaque apparition, dans une fourchette étroite) pour ne jamais sonner comme un enregistrement figé, sans jamais perdre les intervalles qui rendent le thème reconnaissable.

Registre du soutien harmonique (Fa#3, 185Hz) choisi dans la plage déjà validée audible sur haut-parleur de téléphone en V4.2 (165-440Hz) — une octave complète plus bas (110Hz) avait été envisagée puis corrigée avant tout test, car hors de cette plage.

7.2. Validation technique

- **Logique de programmation** (Playwright, horloge virtuelle, 12 minutes simulées) : alternance stricte confirmée entre thème principal (9 oscillateurs : 8 notes + 1 soutien) et phrase réponse (7 oscillateurs), silences réels de 43 à 73 secondes entre apparitions, 12 apparitions observées sur la fenêtre de test. Un premier passage du test s'est révélé faux-positif à corriger : le moteur de jeu déclenche lui-même des effets sonores automatiques (`playPhaseChange`, un seul oscillateur) lors des changements de phase de Réponse mondiale, indépendamment de la musique — ces événements ont dû être exclus du comptage avant que le test ne reflète correctement le comportement réel du système musical.
- **API publique inchangée** (`unlock()`, `setMusicEnabled`, gestion `visibilitychange`, effets sonores) : diff confirmé identique au caractère près par rapport à la V5.1 déjà validée en bêta — aucune régression attendue sur le déverrouillage audio (restrictions d'autoplay), la mise en pause en arrière-plan mobile, ni les réglages Musique/Effets.
- **Volume** : crête de chaque note de mélodie 0.075-0.11, soutien harmonique 0.025, gain racine à 1 — pas de risque de saturation (somme maximale très inférieure à 1.0), et nettement en retrait par rapport aux effets sonores (0.12-0.18) qui doivent rester l'élément le plus marqué.

Limite honnête à documenter (consigne explicite du joueur si Claude Code Cloud ne pouvait pas produire une vraie composition) : cette approche reste de la synthèse en direct par oscillateurs (sinusoïdes/triangles), pas un enregistrement d'instruments réels. Le thème et le mode ont été choisis avec un raisonnement musical explicite (contour mémorisable, mode Dorien pour la couleur émotionnelle), mais le timbre reste électronique et simple par construction. Si ce timbre ne convient toujours pas au ressenti recherché après cette bêta, la vraie limite est celle de la synthèse

procédurale elle-même : une composition avec des timbres d'instruments réels (piano, cordes) demanderait des échantillons audio externes, ce qui sort du principe tenu depuis le début du projet (« aucun asset aux droits incertains »). La meilleure option dans ce cas resterait une intégration externe : un fichier audio fourni par l'utilisateur avec une licence claire, intégré au lecteur déjà en place, plutôt qu'une nouvelle approximation procédurale.

8. Non-régressions vérifiées

Vérification	Résultat
Tests unitaires (node:test)	92/92 verts (86 avant cette phase + 6 nouveaux : migrations V5 → V6 ×2, simulation V5.2 ×2, screens fin de partie, résilience à mobilisation pleine)
Build de production	propre (vite build , aucun avertissement)
PWA	manifest présent, service worker enregistré (vérifié via Playwright sur le build de prévisualisation)
Mobile (390px)	aucun débordement horizontal constaté
Sauvegarde/reprise	partie jouée → rechargement complet de la page → option « Reprendre » bien présente et fonctionnelle (migration V6 comprise)
Réglages (Musique/Effets)	bascules fonctionnelles, sans erreur
Parcours de monétisation (test)	achat/restauration/désactivation Premium (simulation) fonctionnels sans erreur
Erreurs console/page durant tout le parcours	0

9. Déploiement

- Commit moteur/équilibre : 4de7e3f — « RUPTURE V5.2 (1/2) : corrige le plafond de crise bloqué à 98% et le résumé de fin ».
- Commit audio : 8e0b45c — « RUPTURE V5.2 (2/2) : remplace la nappe générative par un thème mélodique fixe ».
- Poussés sur origin/main du dépôt officiel `scuillerjack-oss/rupture-v0`.
- Déploiement GitHub Actions (run [34811501897](#)) : le job `build` a réexécuté `npm test` (vert) et `npm run build` depuis le commit 8e0b45c lui-même (pas depuis une copie locale), puis le job `deploy` (`actions/deploy-pages@v4`) a terminé avec conclusion : `success` à 05:58:02 UTC.

Limite honnête sur la vérification finale : dans cette session, l'accès réseau sortant direct vers `scuillerjack-oss.github.io` (curl comme outil de récupération web) est bloqué par la politique du proxy d'environnement — contrairement aux phases précédentes, la page en production n'a donc pas pu être récupérée et inspectée directement pour comparer le hash exact du

bundle JS servi. La vérification s'appuie ici sur le signal le plus fort disponible : le job `deploy` de GitHub Actions a réussi en partant du build produit par `npm run build` exécuté dans un environnement propre à partir du commit exact `8e0b45c` (pas du `dist local`) — c'est la garantie standard qu'offre `actions/deploy-pages` que ce commit, et aucun autre, est servi. Le build local reproduit par ailleurs le même hash de bundle (`index-CgTwrNke.js`) que celui déjà validé avant ce blocage réseau. Il reste recommandé de confirmer une dernière fois à l'ouverture de la bêta téléphone que le thème musical et le comportement Dangerosité/Résilience corrigés sont bien ceux décrits ici.

10. Points à surveiller lors de la prochaine bêta

1. **Neutralité de la Résilience dans l'archétype « bascule furtive » (§4.5)** : si en jouant à nouveau ce style précis, la Résilience semble ne faire aucune différence perceptible, c'est un comportement connu et documenté, pas un bug — mais un retour précis sur le timing exact de la bascule aiderait à juger si une mécanique dédiée (hors scope ici) devient nécessaire.
2. **Léger mouvement de dangerosite-precocoe** (0% → 6%) et **equilibree-naive** (6% → 0%) : mouvements de faible amplitude à $n=18$ échantillons, plausiblement du bruit statistique plutôt qu'un effet réel du correctif ; à recroiser avec plus de parties si un déséquilibre particulier est ressenti.
3. **Ressenti du thème musical** : le mode Dorien et le contour du thème ciblent directement le retour « trop sombre », mais restent une synthèse électronique simple — un avis après réécoute prolongée en contexte réel de jeu (pas seulement à l'écoute isolée) est le test le plus fiable.
4. **Vérification finale en production** non effectuée par récupération directe de la page (limite réseau de cet environnement, §9) — à confirmer par la bêta elle-même.

Rapport généré à l'issue de la session de travail — RUPTURE reste volontairement plus simple que FRONTIÈRES, aucune sur-architecture n'a été introduite dans cette phase.